



Semester Project

Virtual reality simulation of a flexible joint and an
active suspension system

Author : Kamyar Taher

Professor : Alireza Karimi

PHD student : Philippe Schuchert

Date : Fall semester 2020

Contents

1	Introduction	1
2	Software	1
2.1	Simscape Multibody	1
2.2	SolidWorks	2
2.3	System requirements	3
3	Rotary Flexible Joint	4
3.1	Model to simulate	4
3.2	Dynamics	5
3.3	SolidWorks 3D model	7
3.4	Simulink/Simscape model	10
3.5	Model validation	13
4	Active Suspension System	14
4.1	Model to simulate	14
4.2	Dynamics	15
4.3	SolidWorks 3D model	19
4.4	Simulink/Simscape model	21
4.5	Model validation	24
5	Conclusion	27

1 Introduction

It can be sometimes difficult to imagine how equations are linked to a real-world system, which can make the learning abstract and the overall understanding more complex. This is why in the educational system there is often an emphasis on practical work, as this can have a critical impact on the learning process. This can be even more relevant when the equations are directly applied to move a physical system as in the field of control system.

That is why having visual support while doing theoretical exercise can help to have a better understanding of the problem and allow to observe it from a different perspective. This project was initiated to help bachelor students to better understand the different aspects and effects of controller design. The goal of this project is to develop 3D models for two systems, the *Active Suspension System* and the *Rotary Flexible Joint* and implement them as blocks in *Matlab/Simulink*. It is necessary that the models resulting from this project can be used on the pre-configured computers used by students for the Computer Exercise of the *Control systems and discrete-time control* course.

2 Software

This project requires to run 3D animations in *Matlab/Simulink*. It is therefore common to look for Matlab add-ons that can handle such tasks. Among them, *Simscape* and its extension *Simscape Multibody* stand out. The different 3D models were obtained using *SolidWorks*.

2.1 Simscape Multibody

Simscape allows for the creation of models using physical connections that integrate directly into block diagrams within the *Matlab/Simulink* environment. In addition, *Simscape Multibody* provides a multibody simulation environment for 3D mechanical systems. The working principle of *Simscape Multibody* can be explained from 3 main type of blocks, shown in Figure 1.

- The *Solid* blocks allow for the definition of 3D solids either by defining them directly on *Simscape* or importing them from external files. Parameters such as weight or inertia can be set in order to contribute to the dynamics of the overall system.
- The *Transform* blocks allows for the definition and modification of reference axes. It is possible to build assemblies by linking the axes of several solids.
- The *Joint* blocks allows for the constraining of various degrees of freedom (DoF) between two reference axes (and therefore two solids) depending on the kind of joint used. Several parameters influencing the dynamics can be set, like spring stiffness or damping coefficient. These joints can be actuated by feeding an input signal to them. Two ways of actuation are possible: by *Force/Torque* or by *Motion* (position or angle). Finally, it is also possible to measure various quantities: Position, speed, angle, etc...

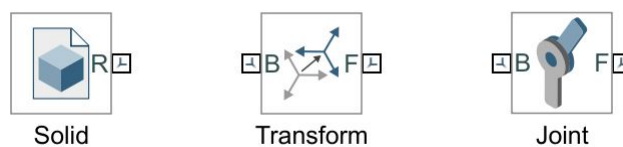


Figure 1: Principal *Simscape Multibody* blocks. From left to right: *Solid* block, *Transform* block and *Joint* (*Revolute Joint*) block.

2.2 SolidWorks

Simscape Multibody is not primarily designed to create 3D solids, but rather to produce realistic simulations of systems from solid parts. It is more efficient to use third-party CAD software to create the 3D model. There are two ways to import 3D parts to *Simscape*. The first method is to import the 3D parts one by one. The second option is to directly import an assembly. Since the block assembly proposed by *Simscape* can quickly become complex and difficult to modify, it is more judicious to use the second method and to build the assembly on the third party CAD software. This will reduce the working time and facilitate eventual modifications and debugging. The export of an assembly cannot be done directly from a CAD software to *Simscape*: it is necessary to have an add-on to make a conversion. This add-on is called the *Simscape Multibody Link Plug-In*. One of the CAD software that is supported for exporting assemblies to *Simscape* is *SolidWorks*. Having an user-friendly interface, *SolidWorks* will be the software of choice used to create the 3D assemblies.

Nevertheless, some good practices need to be taken into account in *SolidWorks* in order for the export of the model to be correctly recreated, and not end up with a complex block diagram with redundant DoF as in the Figure 2.

- It is necessary to form what we call *sub-assemblies* in *SolidWorks*, made up of parts that are constrained between them in order to have correctly defined blocks in *Simscape*. This is the easiest way to ensure that all parts are correctly constrained and that no redundant joints will appear in the exported file. If this method is not used, even a fully constrained model in *SolidWorks* can lead to a model with free joints in the *Simscape* model. These are actually joints that do not result in additional DoF because they are restricted by other constraints.
- It may be useful to transform some pieces within a sub-assembly into a single solid parts. This will allow to lighten the block diagram and improve simulation time of the *Simscape* simulation. But all sub-assemblies should not be completely transformed into a single solid part, because even though *SolidWorks* allows for several different colours to be applied to the same solid part, *Simscape* does not. If we transform our sub-assembly into a single part, it will be in a single color, making it aesthetically unappealing and difficult to differentiate the existing parts.

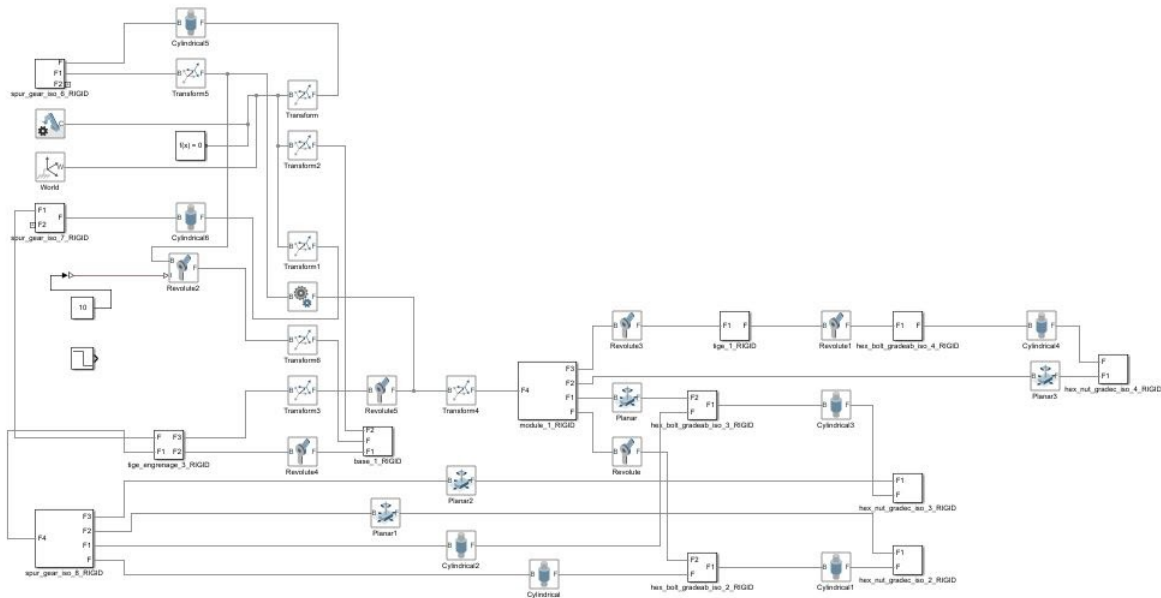


Figure 2: *Simscape* diagram obtained with a badly designed *SolidWorks* model.

By following these recommendations, we end up with a much cleaner block diagram shown in Figure 3. We can see that each subsystem is composed of *Transform* blocks and *Solid* blocks, as shown in Figure 4, for the *gear* subsystem. This diagram is much easier to understand. Even if it still requires some adjustments, they can easily be done due to the simplicity of the diagram.

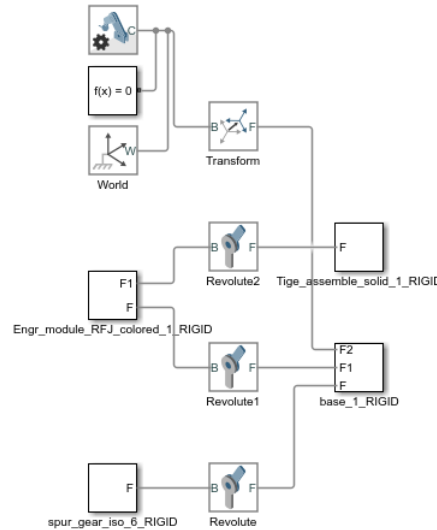


Figure 3: Same *Simscape* assembly as in Figure 2, but obtained from a *SolidWorks* model built according to the recommendations.

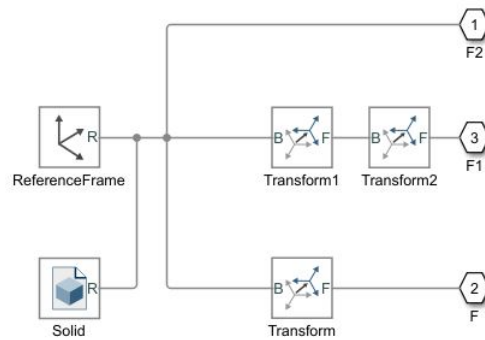


Figure 4: "spur gear iso 6 RIGID" subsystem obtained from the *SolidWorks* export.

2.3 System requirements

Although multiple software and extensions have been used to design the models, the only ones required for the end-user to run the *Simulink* simulations are the *Simscape* and *Simscape Multibody* add-ons. It is also necessary to have a *Matlab* version greater or equal to 2018b.

3 Rotary Flexible Joint

3.1 Model to simulate

The first system that was modelled, shown in Figure 5, is the *Rotary Flexible Joint* from *Quanser*. This system is composed of a main base, where a DC motor drives a gear train. The upper gear is attached to a module with the help of two screws. The module follows the same rotation as this gear. On this module there is a bar that can rotate. Two springs are connected to the module on one side and to the bar on the other side, using screws. This spring configuration allows to softly constraint the bar so it oscillates around the angle of rotation of the module when moving. At rest, the bar is aligned with the module. As shown in Figure 6, the output of the system is defined as the sum of the angle of the main module θ and the angle of the bar α . The input is the voltage applied to the system.

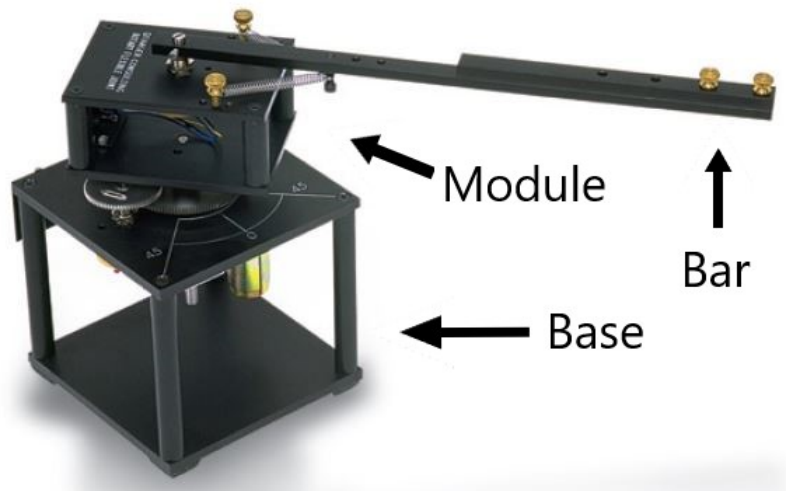


Figure 5: Physical model of the *Rotary Flexible Joint*, taken from [1].

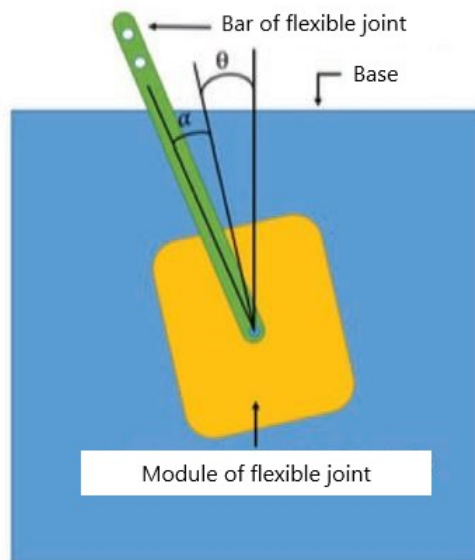


Figure 6: Top view of the system showing the outputs: the angle of the main module θ and the angle of the bar α . Taken from [1].

3.2 Dynamics

In this section we will establish the transfer function (TF) that describes the system between the output $y(t) = \theta(t) + \alpha(t)$ and the input $u(t)$. The equations and numerical values described in this section are taken from the "Control of a Flexible Joint" PDF of the *Control systems and discrete-time control* course [1].

The system can be represented as the diagram shown in Figure 7.

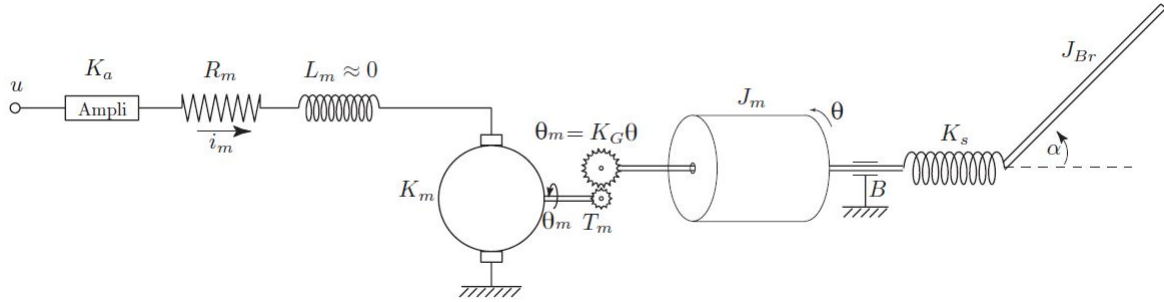


Figure 7: System diagram of the *Rotary Flexible Joint*. Taken from [1].

We can describe the electrical part of the system by using Kirchhoff's voltage law:

$$K_a u(t) = R_m i_m(t) + E_{emf}(t). \quad (1)$$

From the first equation and with $E_{emf}(t) = K_m \dot{\theta}_m(t)$, we can extract the armature current:

$$i_m(t) = \frac{K_a u(t) - K_m \dot{\theta}_m(t)}{R_m}. \quad (2)$$

By applying Newton's law to the motor shaft, we obtain:

$$J_{mot} \ddot{\theta}_m(t) + \frac{T_L(t)}{K_G} = T_m(t), \quad (3)$$

Where J_{mot} as the motor inertia, $T_m(t) = K_m i_m(t)$ as the torque of the motor and $T_L(t)/K_G$ the torque of the load as seen by the gear. The two springs connected to the bar of the system generate two forces that act on the bar anchor points. These two forces will create a torque on the joint which is nonlinear with respect to α , however this relation can be linearized for small α :

$$T_s(t) = K_s \alpha(t), \quad (4)$$

Where K_s is the equivalent stiffness of the flexible joint. Newton's second law for the rotating body around the joint gives us:

$$J_{mod} \ddot{\theta}(t) + b \dot{\theta}(t) + J_{Br} (\ddot{\theta}(t) + \ddot{\alpha}(t)) = T_L. \quad (5)$$

For the arm, we obtain:

$$J_{Br} (\ddot{\theta}(t) + \ddot{\alpha}(t)) + K_s \alpha(t) = 0, \quad (6)$$

Where $\theta_m = K_G \theta$, b the viscous damping of the joint, J_{mod} the inertia of the module and J_{Br} the inertia of the arm. We can also describe the total equivalent inertia at the load as $J_m = J_{mod} + K_G^2 J_{mot}$. Now we can apply the Laplace transform to Eq. 5:

$$(J_{mod} s^2 + bs) \theta(s) + K_s \frac{J_{Br} s^2}{(J_{Br} s^2 + K_s)} \theta(s) = T_L(s). \quad (7)$$

We do the same for the Eq. 6:

$$J_{Br}s^2\theta(s) + (J_{Br}s^2 + K_s)\alpha(s) = 0 \Rightarrow \alpha(s) = -\frac{J_{Br}s^2}{(J_{Br}s^2 + K_s)}\theta(s). \quad (8)$$

Finally, we apply the Laplace transform to Eq. 3:

$$K_G J_{mot} s^2 \theta_m(s) + T_L(s) = K_G K_m i_m(s). \quad (9)$$

Substituting with:

$$i_m(s) = \frac{K_a u(s) - K_m s \theta_m(s)}{R_m} \text{ and } \theta_m(s) = K_G \theta(s),$$

this give us:

$$\left[K_G^2 J_{mot} s^2 + J_{mod} s^2 + bs + K_s \frac{J_{Br} s^2}{(J_{Br} s^2 + K_s)} \right] \theta(s) = \frac{K_G K_m}{R_m} [K_a u(s) - K_m K_G s \theta(s)],$$

that we can also write as:

$$\begin{aligned} [(J_{Br} s^2 + K_s) (R_m J_m s^2 + R_m bs + K_m^2 K_G^2 s) + R_m K_s J_{Br} s^2] \theta(s) \\ = (J_{Br} s^2 + K_s) K_G K_m K_a u(s). \end{aligned}$$

Finally, We obtain:

$$G_\theta(s) = \frac{(J_{Br} s^2 + K_s) K_G K_m K_a}{(J_{Br} s^2 + K_s) (R_m J_m s^2 + R_m bs + K_m^2 K_G^2 s) + R_m K_s J_{Br} s^2}, \quad (10)$$

$$G_\alpha(s) = \frac{-J_{Br} K_G K_m K_a s^2}{(J_{Br} s^2 + K_s) (R_m J_m s^2 + R_m bs + K_m^2 K_G^2 s) + R_m K_s J_{Br} s^2}, \quad (11)$$

$$G(s) = \frac{K_s K_G K_m K_a}{(J_{Br} s^2 + K_s) (R_m J_m s^2 + R_m bs + K_m^2 K_G^2 s) + R_m K_s J_{Br} s^2}. \quad (12)$$

At the beginning of the project it was proposed to replicate the physical model without linearization of the springs, after discussions and considering the way *SimScape* works, doing this seemed too time consuming. This is why the dynamics as described in Eq. 12 will be the one that we will try to replicate in the simulation.

Here are the numerical values associated to the system and that should be used for the simulation.

Numerical values:

Gain of power amplifier	$K_a :$	2	$[-]$
Motor resistance	$R_m :$	2.6	$[\Omega]$
Torque constant	$K_m :$	0.00767	$[\text{Nm/A}]$
Gearbox ratio	$K_G :$	70	$[-]$
Motor Inertia	$J_{mot} :$	3.87×10^{-7}	$[\text{kgm}^2]$
Equivalent rotational stiffness	$K_s :$	1.2	$[\text{Nm/rad}]$
Viscous damping	$b :$	0.004	$[\text{Nm/(rad/s)}]$
Module Inertia	$J_{mod} :$	3.944×10^{-4}	$[\text{kgm}^2]$
Arm Inertia	$J_{Br} :$	0.0037	$[\text{kgm}^2]$

3.3 SolidWorks 3D model

We will now see how the model was built using the tools presented in the section 2. As explained, *SolidWorks* is useful to easily create a nice 3D of the model. This 3D model will then be exported to *Simulink* in order to be used for the *Simscape* simulation. The *SolidWorks* 3D model that was designed for the *Rotary Flexible Joint* can be seen in Figure 8. Some simplifications have been done with respect to the real system. It is worth reiterating that what is sought after is to have an overall visual that is similar to the real system and that is aesthetically pleasing, while having the same DoF as the real system. Therefore some elements, such as screws, some holes, and other small details, have been deliberately neglected.

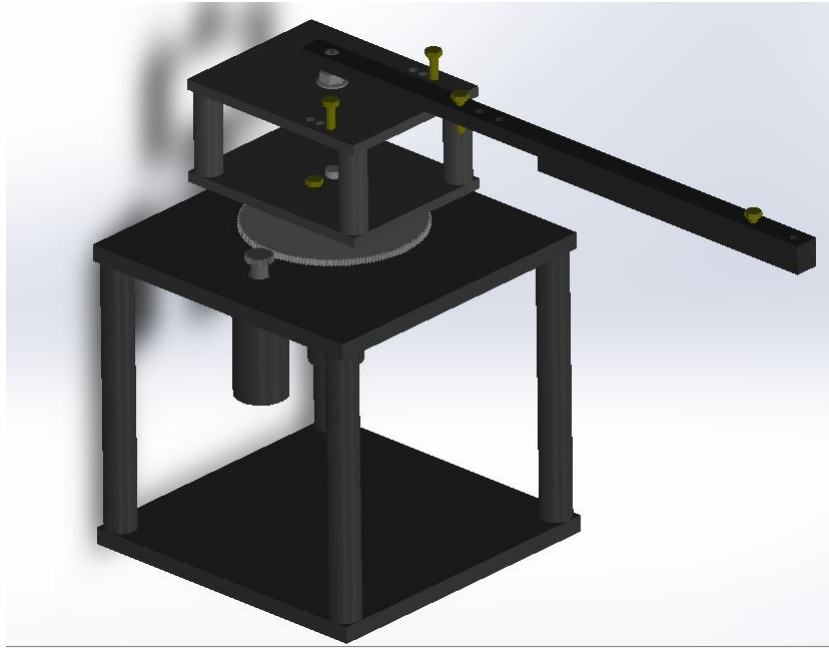


Figure 8: *SolidWorks* 3D model of the *Rotary Flexible Joint*.

Now, as it can be seen from Figure 9, the system is composed of 4 parts shown in Figure 10a, 10b, 10c, 10d. All the elements within a sub-assembly are therefore constrained to each other.

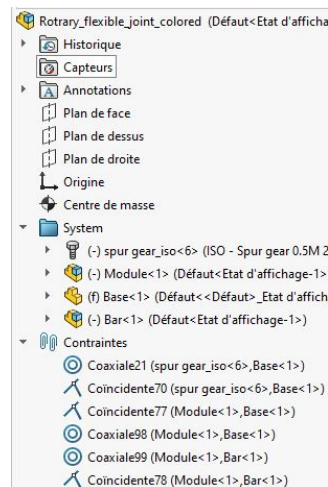
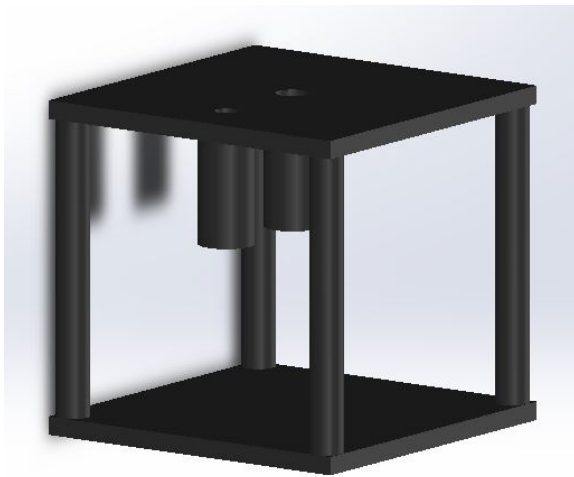
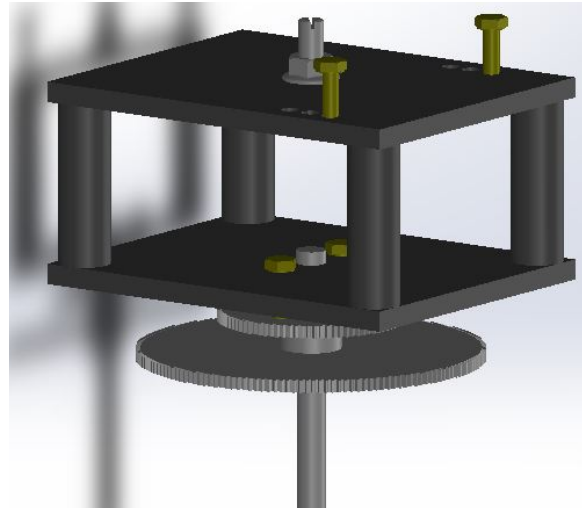


Figure 9: Overview of the different parts and constrains of the *SolidWorks* of the *Rotary Flexible Joint*.
The model is composed of 4 parts: the small gear, the base, the module and the bar.

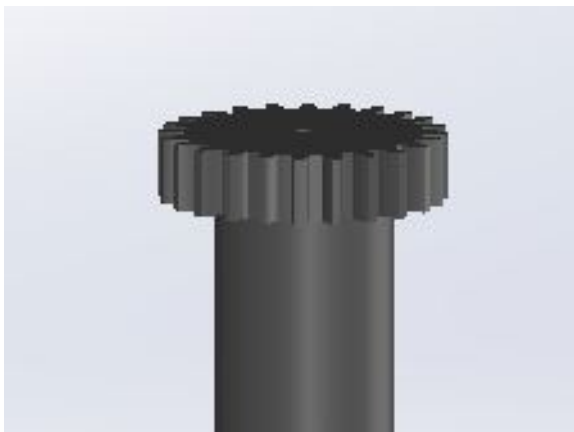
- The first element, shown in Figure 10a, consists only of the base support of the system.
- The second element shown in Figure 10b is a sub-assembly consisting of the central rod, the two central gears, the module, and screws.
- The third element, shown in Figure 10c consists only of the small gear.
- The last sub-assembly, shown in Figure 10d, consists of the bar and two screws. There is one solid part for the bar, and another one that reunite the two screws.



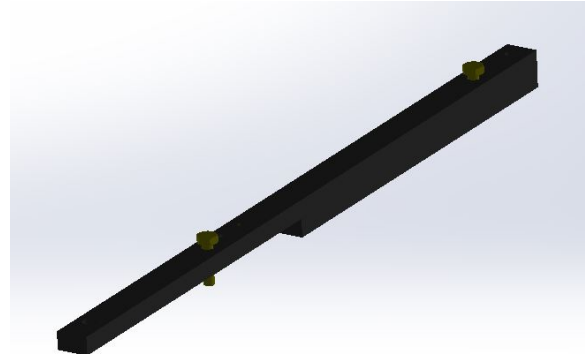
(a) Base.



(b) Module.



(c) Gear.



(d) Bar.

Figure 10: *SolidWorks* sub-assemblies of the *Rotary Flexible Joint* assembly.

Paying attention to the whole 3D model, we notice that the two **springs** that should be linked between the bar and the module are missing. But, as it can be seen in Figure 11, it is possible to model them correctly in *SolidWorks* and constrain them so that a realistic spring movement can be achieved. The springs are actually composed of a multitude of solids with half circle shapes that are connected to each other. However, it is very difficult, if not impossible, to export and manipulate this model correctly in *Simscape*. Furthermore, this come at the cost of a **big computational effort**, making a real-time simulation hardly achievable. This is why they were not included in this model. We will see afterwards an alternative which has been proposed and applied directly to the *Simscape* model.

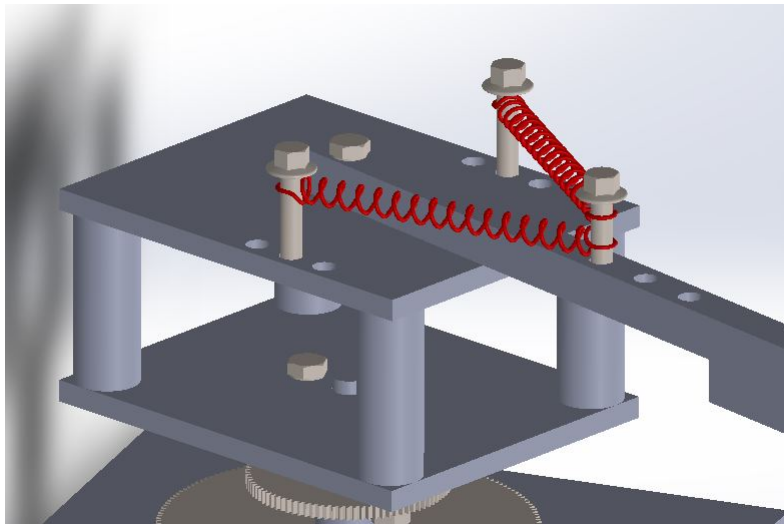


Figure 11: Attempt to model the springs in *SolidWorks*. This model was discarded because it did not worked on *Simscape*.

3.4 Simulink/Simscape model

In this part, we will describe how simulation of the *Rotary flexible joint* has been implemented in *Simulink*. We will start from the top layer and then we will go deeper and see the different part of the implementation.

In Figure 12, we can see the closed-loop system that has been designed for this part of the project. What we are really interested in here, is the sub-system " $G(s)$: *Rotary flexible joint*" which is the top layer block of the implementation of the system. The block acts as a TF, the input is a voltage and the output correspond to the total angle of the system $y(t) = \theta(t) + \alpha(t)$. The system that we want to simulate was shown in Figure 7.

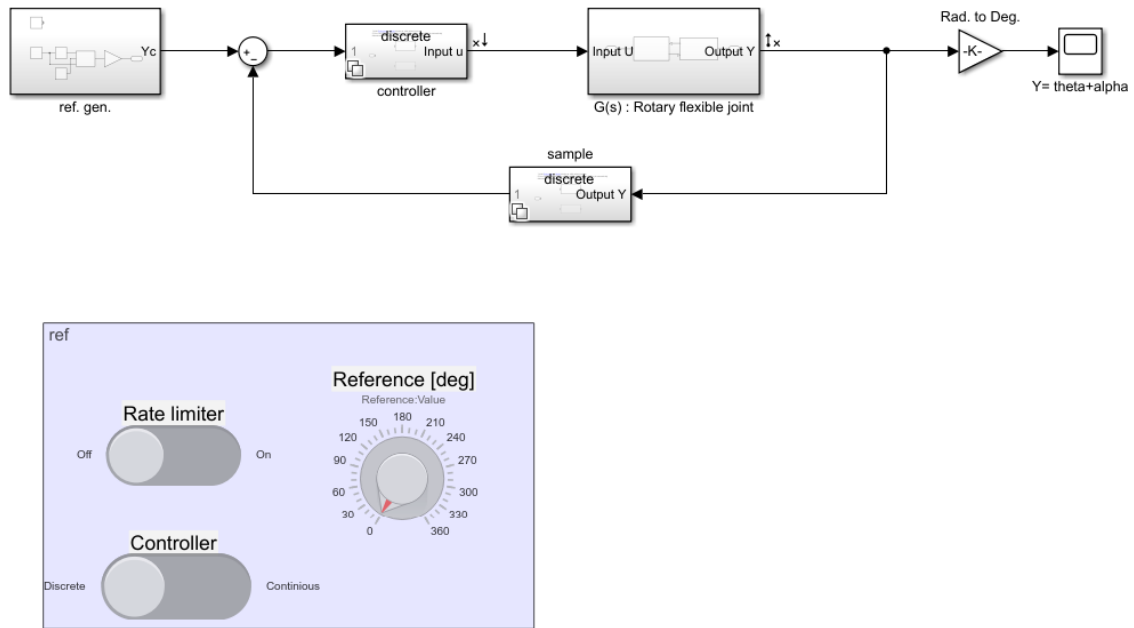


Figure 12: Closed-loop system that was implemented in order to test the block " $G(s)$: *Rotary flexible joint*".

Two blocks are embedded in this block. The first one, *electromechanical*, models the electrical circuit and a portion of the mechanical circuit of the system. The second block, *Multibody*, contains the 3D parts and the simulation of the system.

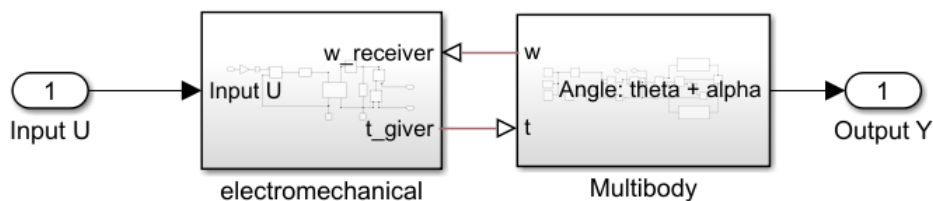


Figure 13: " $G(s)$: *Rotary flexible joint*": First layer of the block.

In the *electromechanical* part, we have the electrical circuit with a gain, a resistance, and a DC motor that allows the conversion from electrical to mechanical circuit. In the mechanical part we have a gear ratio, and a damper simulating the damping applied to the rotating bar (Figure 10d). The damping is done in this block as there is no easy way to link a damper to the ground in *Simscape Multibody*. The last two blocks allow for an interface between the *Multibody* part and the mechanical part to be made.

Without this interface, *Simscape* can't correctly perform the conversion from the 1D mechanical part to the 3D *Multibody* part and will end up distorting the simulation. In order to do so, the torque generated by the *electromechanical* part is sent to the output t_giver to a revolute joint in the *Multibody* part. Then, the angular velocity produced by this torque on the joint is returned from the input $w_receiver$ to the mechanical circuit.

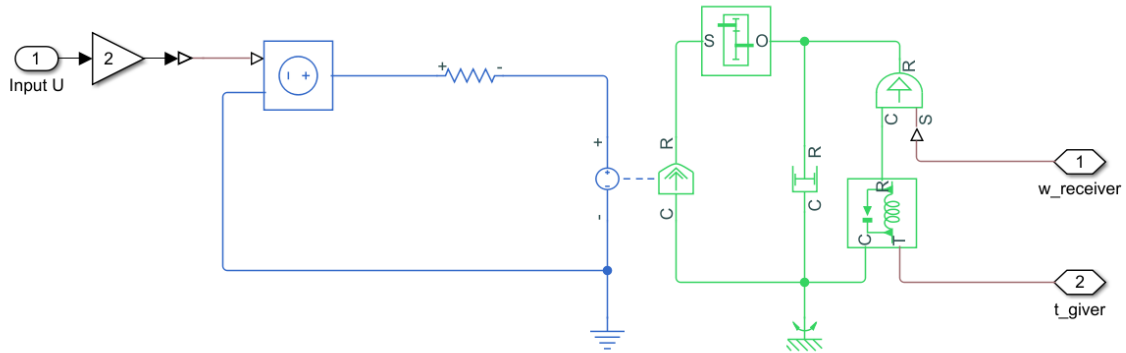


Figure 14: "*electromechanical*": Part of the implementation simulating the electrical circuit, a part of the mechanical circuit and that create an interface with *Simscape Multibody*.

Now, we can look at the heart of the simulation with the Figure 15. Beginning from the left, we have the three initialization blocks. Then we have the *Base* block (Figure 10a), on which two revolute joints are linked. The left revolute joint, named *Revolute*, allows the rotation of the *Gear* (Figure 10b). The movement of this joint is actuated by motion with the angle of *Revolute1*, but with a ratio of 1:7 that is given by the gain block. *Revolute1* is the block that is interfaced with the electromechanical part and that is actuated with a torque. It is the block who actuate the whole Multibody system and it allows the rotation of the *Module* part (Figure 10c). The *Module* block is then linked to *Revolute2* that acts as a rotational spring, which allows the rotation of the *Bar* (Figure 10d).

The two output angles are obtained in Radians from the outputs q from *Revolute1* for θ and *Revolute2* for α .

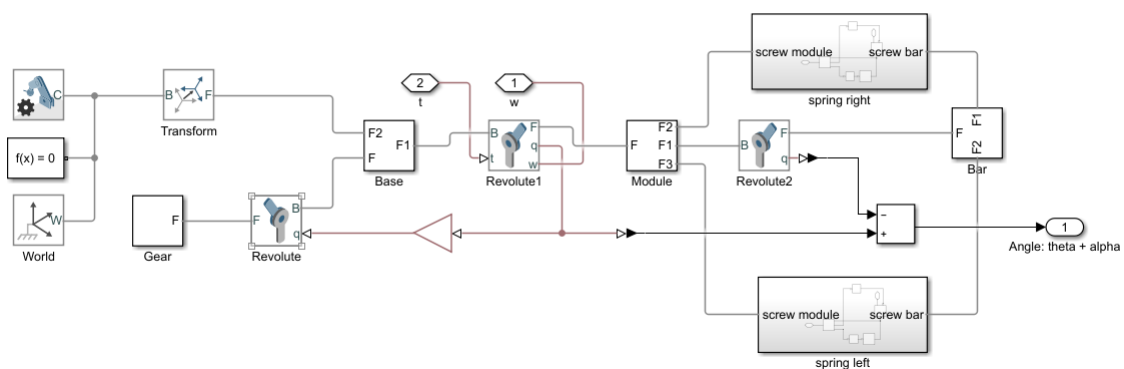


Figure 15: "*Multibody*": Part of the implementation that manage the 3D part and the simulation of the system.

Finally we can take a closer look to the *spring left* and *spring right* blocks.

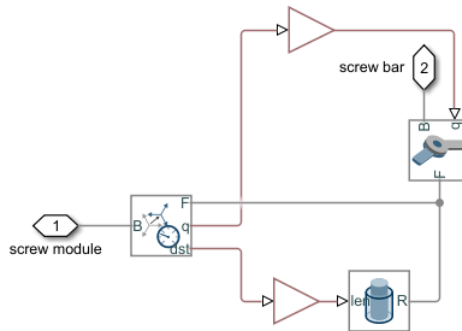


Figure 16: The spring subsystem.

The springs are in fact variable cylinder blocks that can continuously change their length during the simulation. They are connected to the screw that is on the *Bar* via a revolute joint. This joint is actuated by motion and takes as input the difference of angle between the screw on the *Bar* and one of the two screws of the *Module*, depending on which spring it is. So when the *Bar* is rotating relatively to the *Module*, the joint is given the angle that allows to correctly realign the cylinder with his corresponding screw. Then the distance between the screws is sent to the variable cylinder block in order for it to have the appropriate length.

Figure 17 shows the final 3D model that will be viewed during the simulation. Unlike the *SolidWorks* model, this one includes two *Variable Cylinder Solids* which act as the springs.

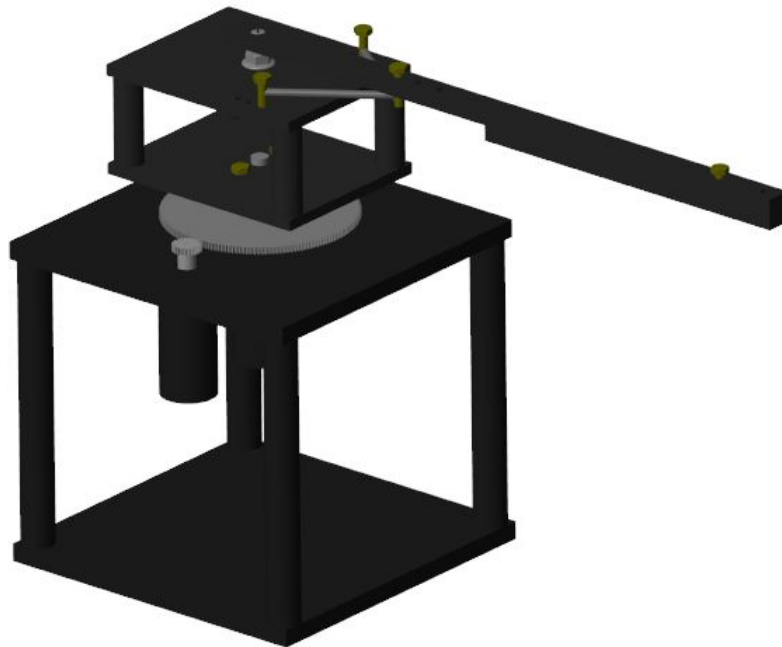


Figure 17: 3D model of the *Rotary flexible joint* that can be viewed in *Matlab* during the simulation.

3.5 Model validation

It is necessary to verify that the system built in *Simscape* actually corresponds to the theoretical model that we want to simulate. For this, the **model linearizer tool** has been used in order to obtain the Bode diagram of the system. It has been compared with the Bode diagram obtained by using the equation 12 of the theoretical model. This comparison is shown in Figure 18.

As we can see, the magnitude and the phase of the two systems are completely superimposed. Therefore, we have a simulation that correctly follows the dynamics of the desired model.

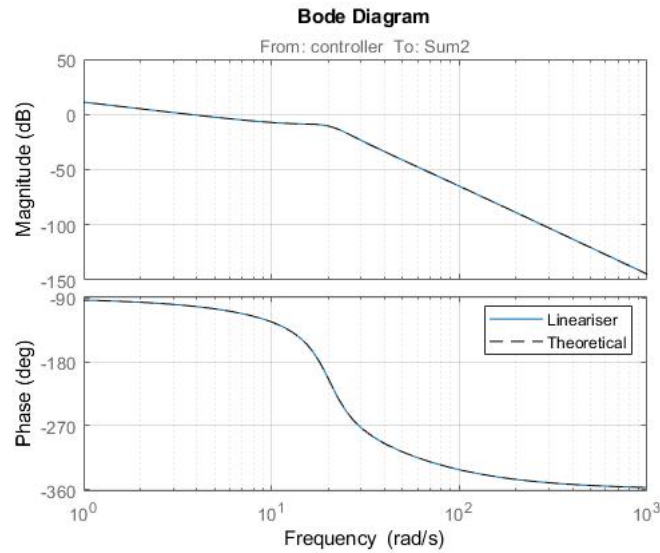


Figure 18: Bode diagram of the theoretical and the simulated model of $G(s)$. The two diagrams are identical.

4 Active Suspension System

4.1 Model to simulate

The second modelled system, shown in Figure 19, is the *Active Suspension System* from Quanser. This system reproduces the dynamics of an active suspension system that can be found in a car. A motor actuates the first plate, which is considered as the tire of the car. From this plate, springs will then transmit the movement to the two plates above. Then, the damping of the two plates can be controlled with the help of a cable that is linked to the two plates and whose rigidity can be modified by a motor that can wrap the cable around a bar.

The system can be represented as shown in Figure 20. It is a MIMO system with two inputs and two outputs. The outputs of the system are the positions X_1 and X_2 of respectively the middle and the upper plate. The inputs are the force F_c and the position Z_r .

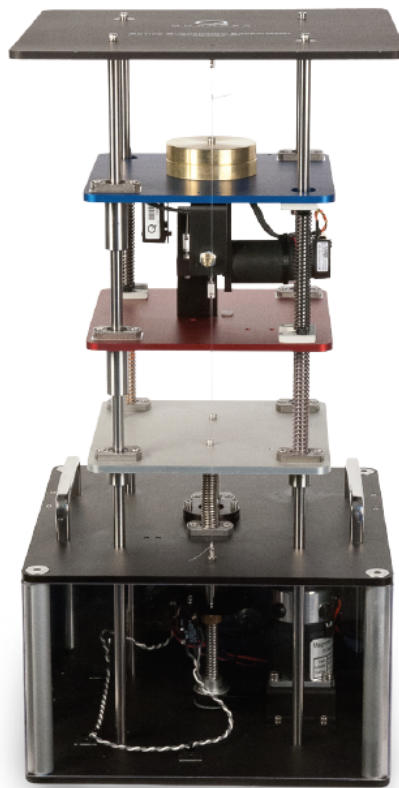


Figure 19: Physical model of the *Active Suspension System*, taken from [2].

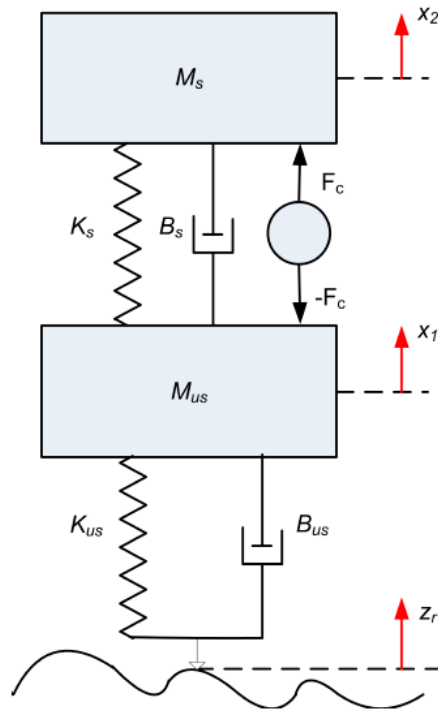


Figure 20: System diagram of the *Active Suspension System*, taken from [2].

4.2 Dynamics

In this section we will establish the 4 TFs corresponding to combinations of the 2 inputs and the 2 outputs that describe the system. The equations and numerical values described in this section are taken from the "Laboratory guide: Active Suspension Experiment for MATLAB/Simulink Users" PDF [2].

We can divide the problem for each of the two masses. We can see the body diagram corresponding to M_s , in Figure 21.

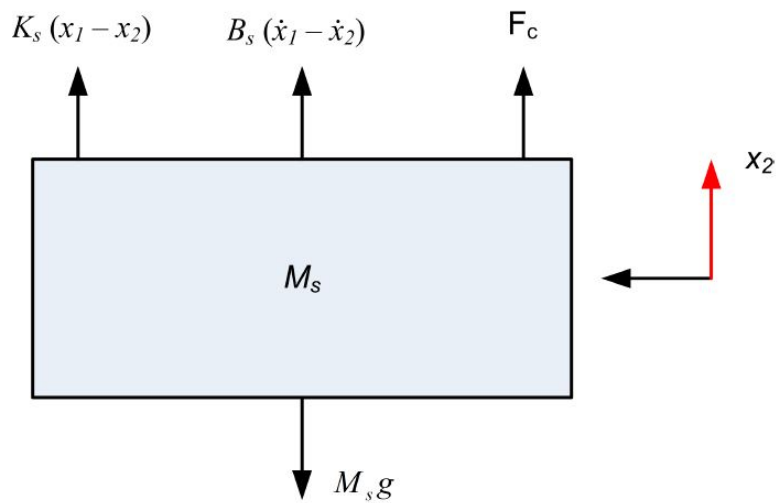


Figure 21: System diagram of the upper plate, taken from [2].

The equations of motion (EOM) of this first system is:

$$\ddot{x}_2 = -g + \frac{F_c}{M_a} + \frac{B_s \dot{x}_1}{M_a} - \frac{B_s \dot{x}_2}{M_s} + \frac{K_s x_1}{M_a} - \frac{K_s x_2}{M_s}. \quad (13)$$

Similarly, we have the body diagram corresponding to M_{us} , in Figure 22.

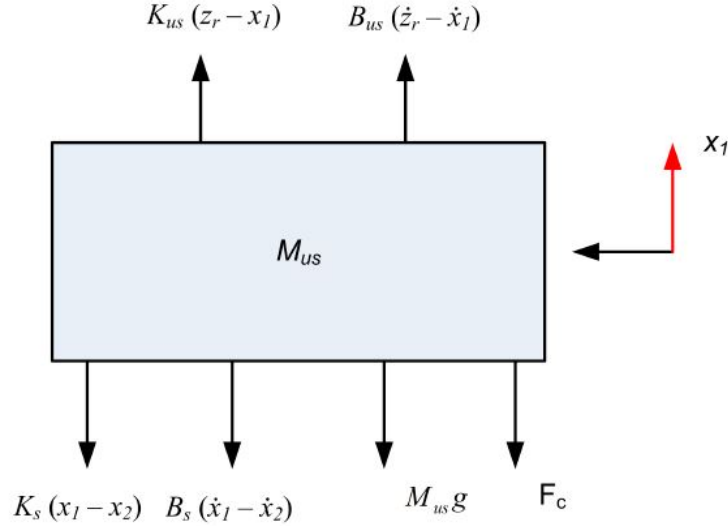


Figure 22: System diagram of the middle plate, taken from [2].

The EOM for this diagram is:

$$\begin{aligned} \ddot{x}_1 = & -g - \frac{F_c}{M_{us}} - \frac{(B_s + B_{us}) \dot{x}_1}{M_{us}} + \frac{B_s \dot{x}_2}{M_{us}} + \frac{B_{us} \dot{z}_r}{M_{us}} \\ & - \frac{(K_{us} + K_s) x_1}{M_{us}} + \frac{K_s x_2}{M_{us}} + \frac{z_r K_{us}}{M_{us}}. \end{aligned} \quad (14)$$

We will prove that the gravity force only changes the equilibrium points and does not affect the dynamics of the system.

At equilibrium $x_1 = x_{eq1}$, $x_2 = x_{eq2}$, the derivative of x_1 and x_2 of any order are zero. The road surface z_r and all its derivatives and the control force F_c are also zero. At equilibrium Eq. 13 and 14 give us:

$$\begin{aligned} K_s x_{eq1} - K_s x_{eq2} + M_{us} g + x_{eq1} K_{us} &= 0 \\ M_s g + K_s x_{eq2} - K_s x_{eq1} &= 0. \end{aligned} \quad (15)$$

Then, the equilibrium points due to gravity become:

$$x_{eq1} = -\frac{g(M_s + M_{us})}{K_{us}}, \quad (16)$$

$$x_{eq2} = -\frac{g(M_s K_{us} + K_s M_s + K_s M_{us})}{K_{us} K_s}. \quad (17)$$

We apply the following change of variable in order to remove gravity forces from the equations:

$$\begin{aligned} x_1 &= z_{us} - \frac{g(M_s + M_{us})}{K_{us}}, x_2 = z_s - \frac{M_s g}{K_s} - \frac{g(M_s + M_{us})}{K_{us}} \\ \dot{x}_1 &= \dot{z}_u, \dot{x}_2 = \dot{z}_s \\ \ddot{x}_1 &= \ddot{z}_{us}, \ddot{x}_2 = \ddot{z}_s. \end{aligned} \quad (18)$$

By substituting Eq. 18 into Eq. 13 and 14 we obtain:

$$M_{us}\ddot{z}_{us} = -B_s\dot{z}_{us} - B_{us}\dot{z}_{us} - F_c + B_s\dot{z}_s + B_{us}\dot{z}_r - (z_{us} - z_s)K_s - (z_{us} - z_r)K_{us}, \quad (19)$$

$$M_s\ddot{z}_s = B_s\dot{z}_{us} + F_c - B_s\dot{z}_s - (z_s - z_{us})K_s, \quad (20)$$

Where the effect of gravity has been eliminated from the equations. The gravity only shifted the equilibrium point of the system. Since the two EOMs of the *Active Suspension System* are assumed to be linear and time-invariant, they can be written under the state-space representation as follows:

$$\begin{aligned} \dot{x} &= Ax + Bu, \\ y &= Cx + Du. \end{aligned} \quad (21)$$

The four state variables, the two inputs and the two outputs can be expressed as follows:

$$x = \begin{bmatrix} z_s - z_{us} \\ \dot{z}_s \\ z_{us} - z_r \\ \dot{z}_{us} \end{bmatrix}, u = \begin{bmatrix} \dot{z}_r \\ F_c \end{bmatrix}, y = \begin{bmatrix} z_s - z_{us} \\ \ddot{z}_s \end{bmatrix}. \quad (22)$$

Using Eq. 19 and 20, we can define matrices A, B, C, and D as follows:

$$\begin{aligned} A &= \begin{bmatrix} 0 & 1 & 0 & -1 \\ -\frac{K_s}{M_s} & -\frac{B_s}{M_s} & 0 & \frac{B_s}{M_s} \\ 0 & 0 & 0 & 1 \\ \frac{K_s}{M_{us}} & \frac{B_s}{M_{us}} & -\frac{K_{us}}{M_{us}} & -\frac{B_s+B_{us}}{M_{us}} \end{bmatrix}, B = \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{M_s} \\ -1 & 0 \\ \frac{B_{us}}{M_{us}} & -\frac{1}{M_{us}} \end{bmatrix}, \\ C &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{K_s}{M_s} & -\frac{B_s}{M_s} & 0 & \frac{B_s}{M_s} \end{bmatrix}, D = \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{M_s} \end{bmatrix}. \end{aligned} \quad (23)$$

Then we can directly use the following "State-space to transfer function" formula:

$$C(sI - A)^{-1}B + D. \quad (24)$$

Finally, by applying this transformation, we obtain the 4 TFs corresponding to combinations of the two inputs and two outputs of the *Active Suspension System*.

$$\begin{aligned} \mathcal{L}\left(\frac{z_s - z_{us}}{\dot{z}_r}\right) &= -sM_s(K_{us} + sB_{us})/(M_sM_{us}s^4 + (M_sB_{us} + B_sM_{us} + M_sB_s)s^3 \\ &\quad + (M_sK_{us} + B_sB_{us} + K_sM_s + K_sM_{us})s^2 + (K_sB_{us} + B_sK_{us})s + K_sK_{us}), \end{aligned} \quad (25)$$

$$\begin{aligned} \mathcal{L}\left(\frac{z_s - z_{us}}{F_c}\right) &= ((M_{us} + M_s)s^2 + sB_{us} + K_{us})/(M_sM_{us}s^4 + (M_sB_{us} + B_sM_{us} + M_sB_s)s^3 \\ &\quad + (M_sK_{us} + B_sB_{us} + K_sM_s + K_sM_{us})s^2 + (K_sB_{us} + B_sK_{us})s + K_sK_{us}), \end{aligned} \quad (26)$$

$$\begin{aligned} \mathcal{L}\left(\frac{\ddot{z}_s}{\dot{z}_r}\right) &= s(B_s s^2 B_{us} + (K_s B_{us} + B_s K_{us})s + K_s K_{us})/(M_s M_{us} s^4 + (M_s B_{us} + B_s M_{us} + M_s B_s)s^3 \\ &\quad + (M_s K_{us} + B_s B_{us} + K_s M_s + K_s M_{us})s^2 + (K_s B_{us} + B_s K_{us})s + K_s K_{us}), \end{aligned} \quad (27)$$

$$\begin{aligned} \mathcal{L}\left(\frac{\ddot{z}_s}{F_c}\right) &= s^2(s^2 M_{us} + sB_{us} + K_{us})/(M_s M_{us} s^4 + (M_s B_{us} + B_s M_{us} + M_s B_s)s^3 \\ &\quad + (M_s K_{us} + B_s B_{us} + K_s M_s + K_s M_{us})s^2 + (K_s B_{us} + B_s K_{us})s + K_s K_{us}). \end{aligned} \quad (28)$$

Here are the numerical values associated to the system and that should be used for the simulation.

Numerical values:

Sprung Mass	$M_s :$	2.45	[kg]
Unsprung Mass	$M_{us} :$	1	[kg]
Suspension Stiffness	$K_s :$	900	[N/m]
Tire Stiffness	$K_{us} :$	1250	[N/m]
Suspension Inherent Damping coefficient	$B_s :$	7.5	[Nsec/m]
Tire Inherent Damping coefficient	$B_{us} :$	5	[Nsec/m]

4.3 SolidWorks 3D model

The *SolidWorks* 3D model that was designed for the *Active Suspension System* can be seen in Figure 23. As for the 3D of the *Rotary Flexible Joint*, we can see that some simplifications have been applied. Some geometries, screw and other minor details have been neglected.

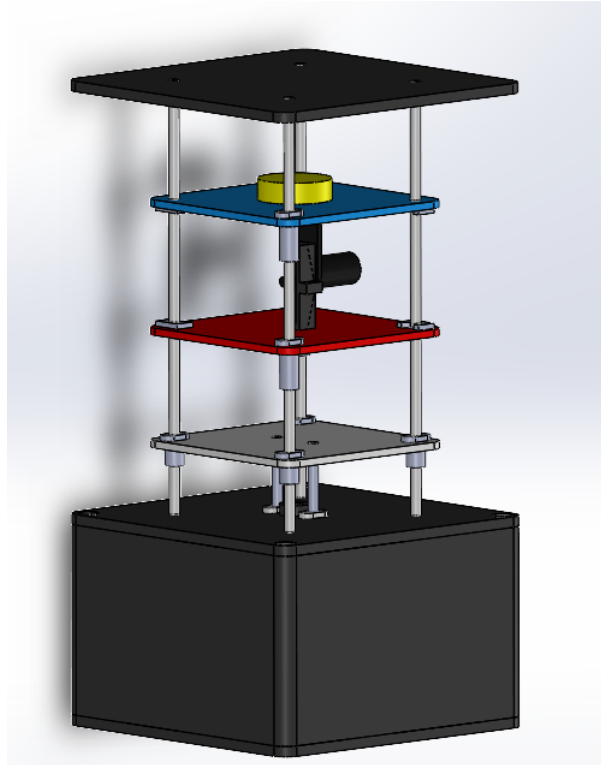


Figure 23: *SolidWorks* 3D model of the *Active Suspension System*.

As it can be seen from Figure 24, the system is composed of 4 parts presented in Figure 25a, 25b, 25c, 25d.

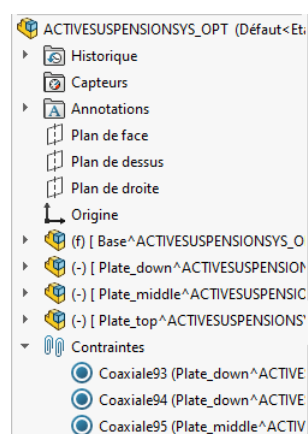
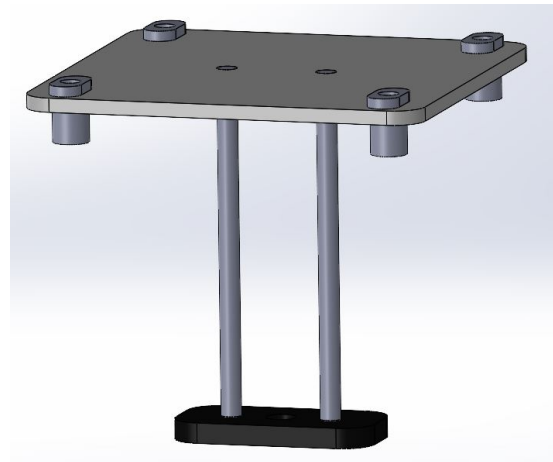


Figure 24: Overview of the different parts and constraints of the *SolidWorks* of the *Active Suspension System*. The model is composed of 4 parts: The base, the lower plate, the middle plate and the upper plate.

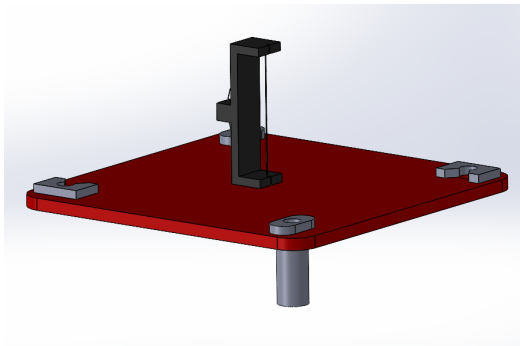
- The first sub-assembly is the base of the system and is composed of all non moving parts. The black box and the black plate are considered as one solid part. All the other pieces are considered as a second solid part.
- The second sub-assembly is the lower plate of the system. The plate is considered as one solid part. All the other pieces are considered as a second solid part.
- The third sub-assembly is the middle plate of the system. There is one solid part for the red plate, one for the lower part of the motor mechanism and a final solid part uniting all the other elements.
- The last sub-assembly is the upper plate of the system. There is one solid part for the blue plate, one for the motor, one for the yellow mass, and a final part uniting all the others elements.



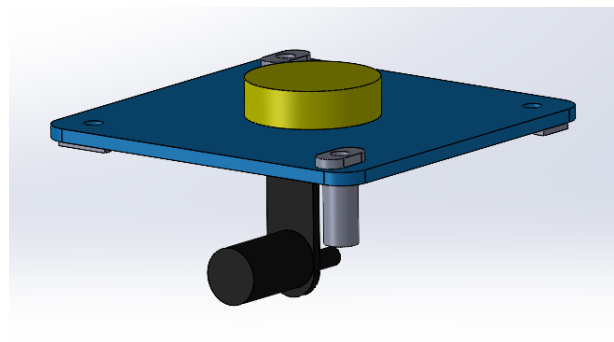
(a) Base.



(b) Lower plate.



(c) Middle plate.



(d) Upper plate.

Figure 25: *SolidWorks* sub-assemblies of the *Active Suspension System* assembly.

As for the *Rotary Flexible Joint*, the *SolidWorks* 3D of this model does not contain the springs. As we will see, they were directly implemented in the *Simscape* model.

4.4 Simulink/Simscape model

In this part, we will describe how the simulation of the *Active Suspension System* has been implemented in *Simulink*. As for the *Simulink* of the *Rotary Flexible Joint*, we will start from the top layer block and then we will go into lower sub-layers of the system. In Figure 26, we can see the closed-loop system that has been designed to test this simulation. The top layer block of the implementation of the system is "*G(s): Active Suspension System*". The block acts as a TF, the inputs are the force F_c and the reference position Z_r and the outputs are $\ddot{Z}_s$ and $Z_s - Z_{us}$. The system that we want to simulate was shown in Figure 20.

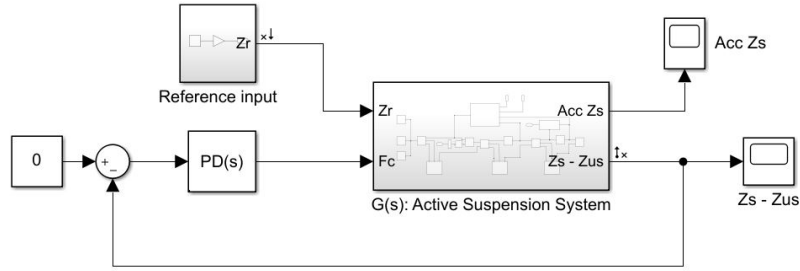


Figure 26: Closed-loop system implemented in order to test the block "*G(s): Active Suspension System*".

In the "*G(s): Active Suspension System*" sub-system, we can directly find the *Multibody* implementation, shown in Figure 27. Beginning from the left, there are the 3 initialization blocks, followed by the *Base* part of the system (Figure 25a). From the *Base*, two outputs are used for the subsystem "*Lower Springs*". The other output is sent to the "*Sensors*" subsystem and to the *Prismatic Joint*. This first joint replicates the displacement Z_r of the *Lower plate*, it is actuated by motion with the reference position Z_r . This joint is then linked to the *Lower plate* (Figure 25b), which is linked to the second *Prismatic Joint*. This joint replicates the dynamics of K_{us} and B_{us} . Then, a part of the output of this joint is used for the *Sensors* and the other part is linked to the *Middle plate* (Figure 25c). This plate contains the mass M_{us} and represents the first reference position X_1 . This plate is also linked to a last *Prismatic Joint*, that replicates the dynamics of K_s and B_s . This joint is also linked to the *Sensors* and to a last *Solid part* block, named *Upper plate* (Figure 25d), that simulates the mass M_s and the reference position X_2 .

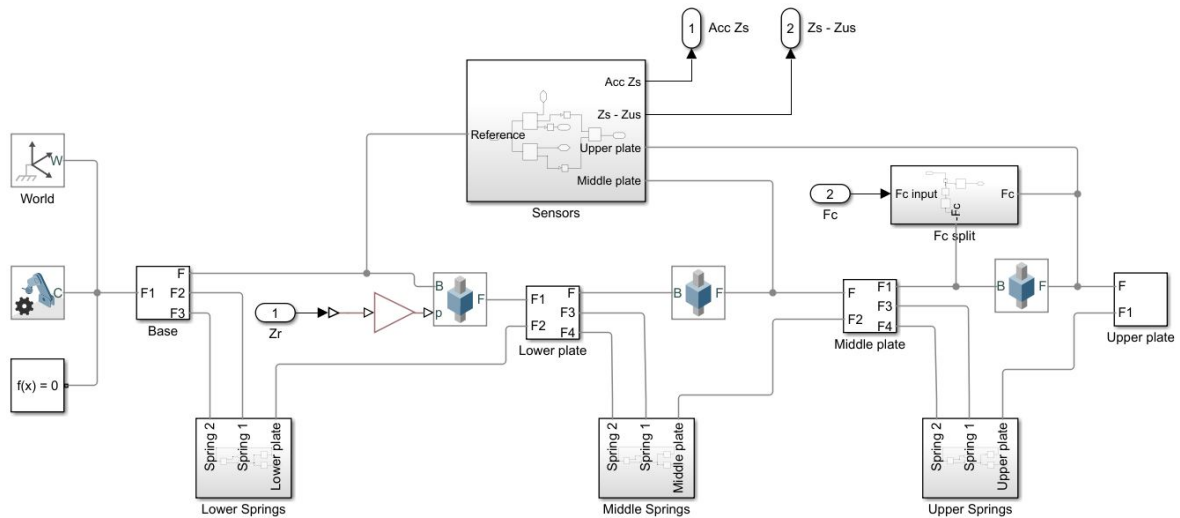


Figure 27: "*G(s): Active Suspension System*": Whole *Multibody* implementation.

The F_c split subsystem, shown in Figure 28, transforms the F_c input signal to forces F_c and $-F_c$ that are respectively applied to the Upper and to the Middle plate.

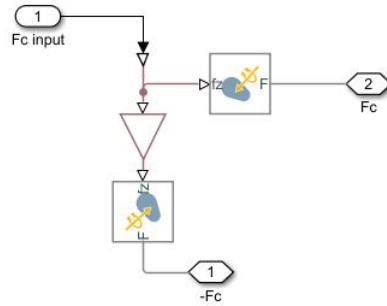


Figure 28: " F_c split": Subsystem creating forces F_c and $-F_c$ from an input signal.

The three Springs subsystems are all identical. The *Middle Springs* subsystem is shown in Figure 29. They consist of two *Variable Cylindrical Solid* blocks that can change length during simulation. The length of the springs are given via a *Transform Sensor* blocks that gives the distance between the two corresponding plates.

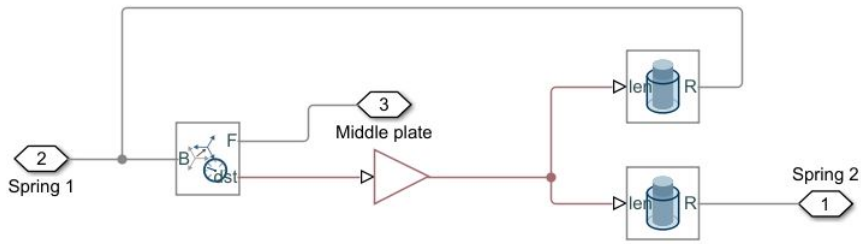


Figure 29: "*Middle Springs*": Subsystem of the springs

Finally, in Figure 30, there is the *Sensors* subsystem. It allows to get the values of $\ddot{Z}_s$ and $Z_s - Z_{us}$ with the help of *Transform Sensor* blocks.

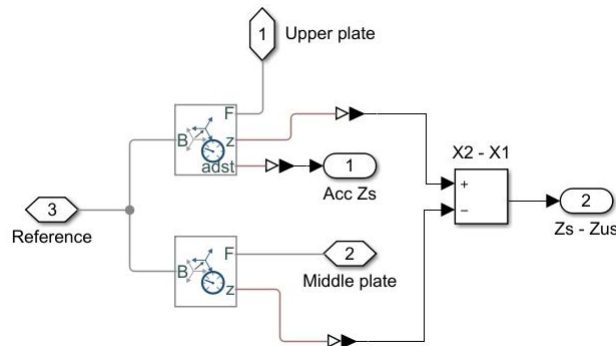


Figure 30: "*Sensors*": Subsystem regrouping the sensors needed to compute all the outputs.

In Figure 31 we can see the final 3D model that will be viewed during the simulation. This model includes 6 *Variable Cylinder Solids* that act as springs.

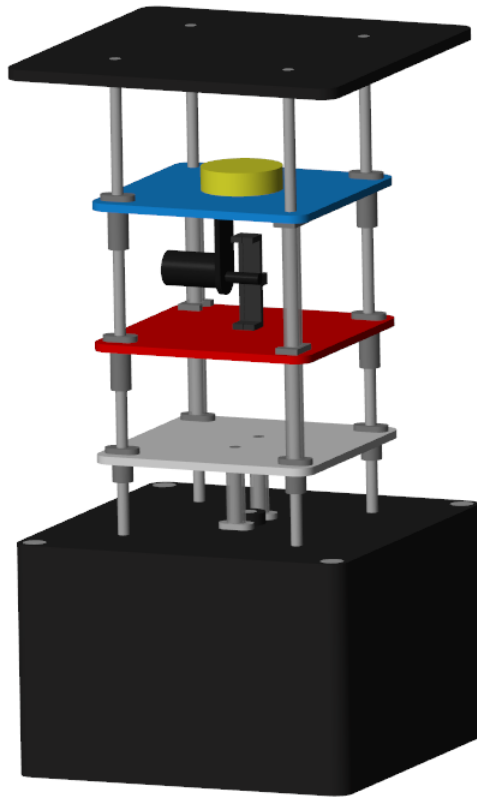


Figure 31: 3D model of the *Active Suspension System* that can be viewed in *Matlab* during the simulation.

4.5 Model validation

For the Active Suspension System, since it is a MIMO system, 4 TFs must be checked in order to completely validate the model. The TFs that need to be validated are those from Eq. 25, 26, 27 and 28. Once again the Bode diagram were obtained thanks to the *model linearizer* tool. We will see that for this simulation, not all the TFs could exactly be replicated.

Let's begin by looking at the Bode diagram of the Figure 32 and 33 that represent respectively the Bode diagram of $\mathcal{L}\left(\frac{z_s - z_{us}}{F_c}\right)$ and $\mathcal{L}\left(\frac{\ddot{z}_s}{F_c}\right)$. We see that for these TFs, the *Simscape* model replicates exactly the theoretical equations.

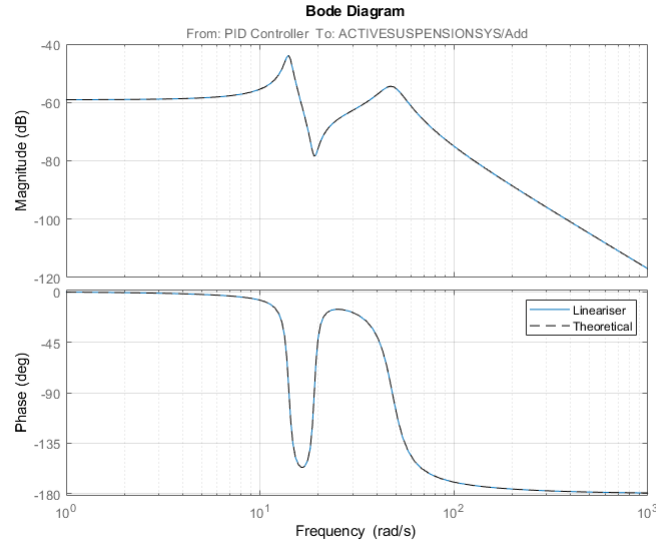


Figure 32: Comparison of Bode diagram of the simulated and the theoretical TF: $\mathcal{L}\left(\frac{z_s - z_{us}}{F_c}\right)$.

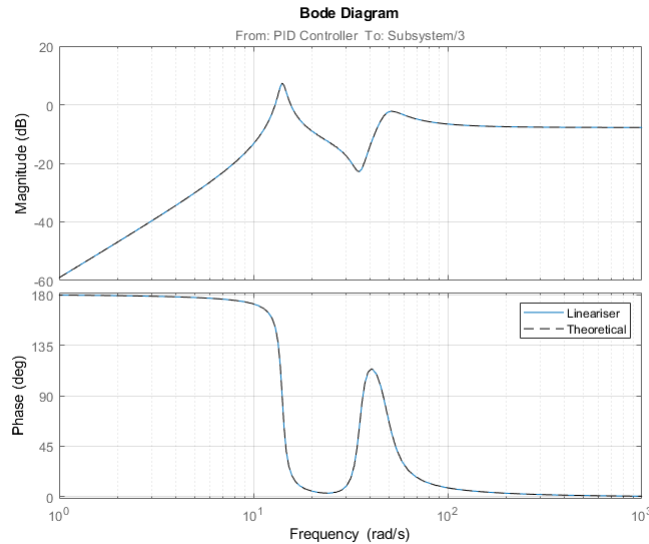


Figure 33: Comparison of Bode diagram of the simulated and the theoretical TF: $\mathcal{L}\left(\frac{\ddot{z}_s}{F_c}\right)$.

However, this is not the case for the two other TFs. Indeed, by looking at Figure 34 and 35, we find that there is a difference between the theoretical and simulated model. This is taking into account the fact that the theoretical model takes as input the speed of the plates $\dot{z}_r$, while the simulated model takes the position z_r . We are looking at $s * \mathcal{L}\left(\frac{z_s - z_{us}}{\dot{z}_r}\right)$ and $s * \mathcal{L}\left(\frac{\ddot{z}_s}{\dot{z}_r}\right)$ in order to have z_r as input for both theoretical and simulated model.

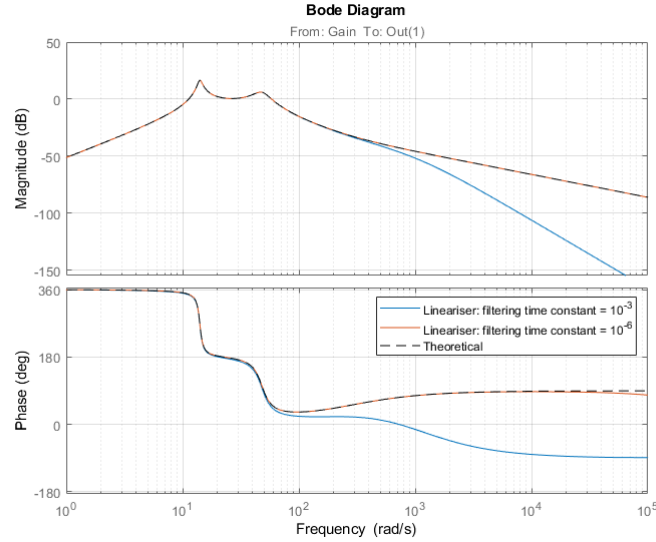


Figure 34: Comparison of Bode diagram of the simulated and the theoretical TF: $s * \mathcal{L}\left(\frac{z_s - z_{us}}{\dot{z}_r}\right)$. For the simulation, filtering time constant of 10^{-3} and 10^{-6} were compared.

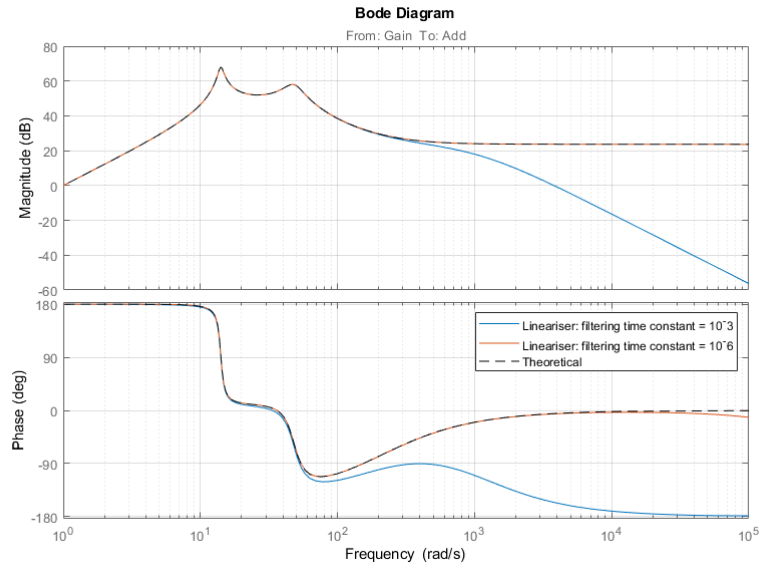


Figure 35: Comparison of Bode diagram of the simulated and the theoretical TF: $s * \mathcal{L}\left(\frac{\ddot{z}_s}{\dot{z}_r}\right)$. For the simulation, filtering time constant of 10^{-3} and 10^{-6} were compared.

We can better understand the source of the problem by taking a closer look at Figure 36. This Bode diagram shows that the difference $G(s)_{\text{simulated}}/G(s)_{\text{theoretical}}$ for both $s * \mathcal{L}\left(\frac{z_s - z_{us}}{\dot{z}_r}\right)$ and $s * \mathcal{L}\left(\frac{\ddot{z}_s}{\dot{z}_r}\right)$ are completely equal and can be represented by low-pass filter with a double pole at 10^3 rad/s . The problem comes from the solver that is unable to directly solve the model from the actuation by motion of the input z_r . It needs time derivatives of the input signals. One way of providing the necessary input derivatives is by filtering the input through a low-pass filter. The error can be reduced by decreasing the *filtering time constant* in the solver of the Simscape model. This will shift the low pass to a frequency in rad/s which is equal to the inverse of the value of the time constant. The comparison of two constant was shown in Figure 34 and 35. However, decreasing the time constant drastically increase the simulation time of the model and this does not longer allow real-time simulation. It was therefore decided to keep the initial constant of 10^{-3} and to accept the associated error occurring at high frequencies.

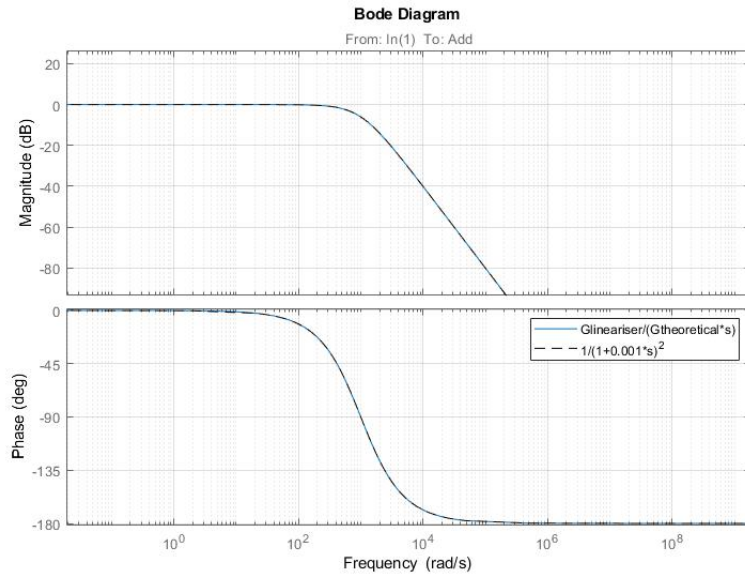


Figure 36: Bode diagram showing that the difference between the simulated and the theoretical model for the TFS, $s * \mathcal{L}\left(\frac{z_s - z_{us}}{\dot{z}_r}\right)$ and $s * \mathcal{L}\left(\frac{\ddot{z}_s}{\dot{z}_r}\right)$, can be described by a low-pass filter.

5 Conclusion

This project allowed the realization of simulations for two physical systems, the *Active Suspension System* and the *Rotary Flexible Joint*. These simulations were designed on a *Simulink* add-on named *Simscape* and its extension *Simscape Multibody*. The 3D models have been designed on *SolidWorks* in order to improve the overall appearance of the models and facilitate the design process before they were exported into *Simscape*. Since *Simscape* is designed to simulate rigid bodies, the implementation of 3D spring models, that should be made for both systems, turned out to be complicated. An alternative with rods that can change their length within the simulation was designed directly on *Simscape* to serve as a spring, allowing a satisfactory final visual result.

For the *Rotary Flexible Joint*, the dynamics of the simulation could exactly replicate those of the theoretical equations, in which the two-springs system has been simplified and linearized into a single spring. For the *Active Suspension System*, as it is a MIMO system, 4 TFs had to be verified in order to validate the model. The dynamics of the simulation could exactly replicates two of the TFs. The two other TFs behave correctly for frequencies lower than 10^3 rad/s, but get errors at higher frequencies due to computation errors. The error could be reduced at the cost of longer computation time, but as we want a real-time simulation, some error had to be accepted.

Finally, the realization of this project should allow bachelor students to have access to visually pleasing simulations with rather realistic dynamics. This should hopefully help them to have another point of view on the control algorithms used in their control systems course and make their practical exercise session more enjoyable.

References

- [1] Computer-based Exercises for Control Systems Course, "*Control of a Flexible Joint*". 2020. PDF file.
- [2] Jacob Apkarian, Amin Abdossalami, Quanser. "*Laboratory guide: Active Suspension Experiment for MATLAB/Simulink Users*". 2013. PDF file.
Link: www.made-for-science.com